Threshold Technical Report

Implementation reference for maintainers, reviewers, and coding agents

This report describes the repository as implemented. It separates verified behavior from limitations and should be used to avoid assumptions.

Threshold Technical Report

Purpose of this document

This document is the technical source of truth for the Threshold repository as currently implemented. It is written for another coding agent, reviewer, or maintainer who needs to understand the project without guessing.

Use this report together with the source code. When this report and the code disagree, the code is authoritative and this report should be updated. Do not infer behavior that is not explicitly described here or verified by the implementation.

Repository identity

- Project: Threshold
- Repository: https://github.com/SujayBWJ/Threshold
- Primary branch: main
- Latest implementation work at the time of this report: live payment flow, Lora links, judging evidence, selectable incident fixtures, dependency security scan, sandbox verification, incident ledger, and capability-selection UI.
- Runtime: Node.js with TypeScript executed through tsx.
- Server framework: Hono with @hono/node-server.
- Frontend: static HTML, CSS, and browser JavaScript served by the same server.

Product definition

Threshold is an agent-oriented capability gateway. A caller agent has a task it cannot complete. Threshold lets it inspect a catalog of capabilities, select a matching provider, pay for one request through x402 on Algorand, forward the incident to the protected provider route, and return a structured result.

The primary demo is a paid capability flow. It demonstrates two real logic incidents plus a dependency security lookup:

- The divide fixture returns multiplication instead of division.
- The regional user cache key uses only userId.
- The same user identifier can exist under more than one region.
- A user cached for us-east can be returned for eu-west.
- The correct fix is to include region and user ID in the cache key.
- The dependency fixture has passing logic tests but a pinned version matched by an OSV-compatible mock security feed.

The provider response contains either a diagnosis, confidence value, unified diff, and verification command, or a structured dependency finding. Bug patches are applied in a temporary verification workspace and tested there; the demo does not modify the user's real repository.

What the project is and is not

It is

- A working prototype of agent capability discovery.

- A working x402 payment flow on Algorand TestNet.
- A real HTTP 402 to signed payment to facilitator settlement to paid retry sequence.
- A live SSE event stream consumed by the browser.
- A structured incident-resolution response backed by Gemini.
- A demo of a server-controlled payer wallet.

It is not

- A production wallet custody system.
- A multi-user authenticated application.
- A general autonomous patch-application sandbox.
- Proof that every AI-generated patch is safe to apply.
- A durable database-backed activity ledger. The browser ledger uses localStorage.
- A MainNet judging configuration. The hackathon path is TestNet.

High-level architecture

```
Browser
  |
  | POST /api/agent/run-payment-flow
  | receives Server-Sent Events
  v
Hono server: server/index.ts
  |
  +--> GET /api/catalog
  |
  +--> POST /api/resolve-incident
  |       protected by x402 paymentMiddleware
  |
  +--> x402ResourceServer
  |       ExactAvmScheme + GoPlausible facilitator
  |
  +--> runIncidentPaymentFlow
          unpaid request -> HTTP 402
          payer signer -> x402 client
          facilitator settlement -> paid retry
          structured provider response -> SSE events
```

The code path used by the terminal script and browser endpoint is shared. The terminal script calls runIncidentPaymentFlow directly. The browser route calls the same function and forwards emitted events into an SSE stream.

Important files and ownership

server/index.ts

Owns server startup and route composition.

Responsibilities:

- Loads .env from repository root and optionally server/.env.
- Requires AVM_ADDRESS, provider addresses, and FACILITATOR_URL.
- Validates Algorand addresses and payment price format.
- Creates HTTPFacilitatorClient with FACILITATOR_URL.
- Creates x402ResourceServer.

- Registers Algorand TestNet and MainNet schemes.
- Registers the Bazaar/discovery extension.
- Defines payment requirements for protected routes.
- Serves static files from server/public.
- Registers the SSE route and protected provider routes.

server/src/config/payment.ts

Owns network and asset selection.

Important behavior:

- X402_NETWORK=testnet selects Algorand TestNet and TestNet USDC ASA 10458941.
- Any value other than testnet or mainnet is rejected by server/index.ts.
- If X402_NETWORK is absent, the configuration helper selects MainNet. For hackathon judging, the local .env must explicitly contain X402_NETWORK=testnet.
- getBasePaymentRequirement returns exact scheme, selected network, pay-to address, and asset.
- Default price is $0.001.

server/src/catalog/apis.ts

Owns the public capability catalog.

Current entries:

1. resolve-incident

- Endpoint: POST /api/resolve-incident
- Capabilities: bug-resolution, patch-generation, incident-debugging

2. code-review

- Endpoint: POST /api/code-review
- Capabilities: code-review, bug-detection, security-analysis

3. summarize

- Endpoint: POST /api/summarize
- Capabilities: text-summarization, summarization, text-processing

The catalog contains public provider names and wallet addresses. It must not contain mnemonics or private keys.

server/src/services/payment/incident-flow.ts

This is the central incident payment implementation. Do not create a second implementation for the browser or terminal.

Responsibilities:

- Owns the incident fixture.
- Emits timestamped step, error, and complete events.
- Calls /api/catalog.
- Selects the first catalog entry containing bug-resolution.

- Records alternative capability IDs and the selection reason.
- Sends the unpaid incident request.
- Requires HTTP 402 from the protected provider route.
- Decodes the payment-required header.
- Uses the advertised network from the 402 requirement when available.
- Creates a signer from the server-side mnemonic.
- Validates the derived signer address against AVM_PAYER_ADDRESS for funded-wallet runs.
- Creates and registers an x402Client and ExactAvmScheme.
- Calls wrapFetchWithPayment for payment construction, signing, facilitator settlement, and paid retry.
- Extracts the settlement response from payment headers.
- Emits real settlement data and the provider response.
- Builds a TestNet or MainNet Lora transaction URL.
- Converts exceptions into an SSE-visible error event.

The seed is copied before crypto key construction because the key helper can mutate the seed buffer. This prevents a false mnemonic/address mismatch.

server/src/routes/run-payment-flow.ts

Owns the browser SSE transport.

Endpoint:

```
POST /api/agent/run-payment-flow
Content-Type: application/json
```

Accepted options:

```
{
  "network": "testnet",
  "wallet": "funded"
}
```

Behavior:

- network=testnet requests TestNet behavior.
- Any other network value results in useTestnet=false at this route boundary; the flow then follows the 402-advertised network.
- wallet=empty creates a random empty wallet for the failure demo.
- wallet=funded uses AVM_MNEMONIC.
- Returns text/event-stream.
- Calls runIncidentPaymentFlow and forwards every event immediately.
- Closes the stream when the flow finishes.

SSE event shape:

```
event: step|error|complete
data: { JSON event object }
```

Event object:

```
{
  "type": "step",
```

```
  "actor": "AGENT A",
  "title": "Incident detected",
  "detail": "...",
  "timestamp": "ISO-8601 timestamp",
  "data": {}
}
```

server/src/routes/resolve-incident.ts

Owns the protected provider handler after payment middleware has verified payment.

It validates:

- non-empty runtime string
- non-empty language string
- object-shaped error
- non-empty files array
- every file has a non-empty string path and string content

It calls generateIncidentResolution and returns:

```
{
  "success": true,
  "resolution": {
    "diagnosis": "string",
    "confidence": 1,
    "patch": [{ "path": "src/file.ts", "diff": "unified diff" }],
    "verification": { "command": "...", "expected": "..." }
  }
}
```

server/src/services/ai/gemini.ts

Owns Gemini access and response validation.

- Reads GEMINI_API_KEY and GEMINI_MODEL from the environment.
- Uses @google/generative-ai.
- Requests JSON for incident resolution.
- Extracts JSON from a response, including fenced JSON.
- Checks diagnosis type, confidence type, patch array, and verification command.
- Converts quota and provider failures into AIProviderError.

Gemini is the AI provider behind the incident API. Gemini is not the payment system. x402 and Algorand handle access/payment; Gemini generates the debugging content after payment middleware permits the provider route to run.

server/public/app.js

Owns browser behavior.

Responsibilities:

- Starts the SSE request from the Run Autonomous Task button.
- Reads streamed chunks with TextDecoderStream.
- Parses SSE data events.
- Adds events to the execution trace.

- Updates the settlement inspector only from event data.
- Renders the provider’s returned diagnosis, unified diff, and verification command.
- Marks the capability selected by the actual catalog event.
- Stores incident ledger records in localStorage.
- Computes average successful resolution cost and time from stored event timestamps.
- Renders a failed ledger record when the stream emits an error.

The browser must never receive AVM_MNEMONIC, private keys, or secret key material.

Exact payment sequence

1. Browser calls the SSE endpoint.
2. runIncidentPaymentFlow emits the incident fixture.
3. Backend fetches GET /api/catalog.
4. Backend selects resolve-incident by bug-resolution capability.
5. Backend POSTs the fixture to /api/resolve-incident without payment.
6. x402 middleware returns HTTP 402 and a payment-required header.
7. Backend reads amount, asset, provider wallet, and network from that requirement.
8. Backend creates the server-side payer signer.
9. Backend verifies the funded payer address matches AVM_PAYER_ADDRESS.
10. Backend registers ExactAvmScheme.
11. wrapFetchWithPayment creates and signs the payment payload.
12. The x402 client sends the payment through HTTPFacilitatorClient.
13. GoPlausible verifies and settles the Algorand TestNet payment.
14. The x402 client retries the original POST with payment proof.
15. The protected route calls Gemini.
16. The provider returns structured diagnosis and patch JSON.
17. Backend extracts the payment-response settlement data.
18. Backend emits paid retry and complete events.
19. Browser updates inspector, patch panel, ledger, and transaction link.

Payment facts for TestNet judging

- Environment variable: X402_NETWORK=testnet.
- Network label: Algorand TestNet.
- TestNet USDC ASA: 10458941.
- Default request price: $0.001.
- On-chain amount: 1000 micro-USDC.
- Facilitator: configured by FACILITATOR_URL, normally https://facilitator.goplausible.xyz.
- Explorer URL: https://lora.algokit.io/testnet/transaction/<transaction-id>.

The .env file is ignored and must never be committed. A clean setup needs a user-provided .env based on .env.example and the README instructions.

Failure behavior

The UI has an Underfunded wallet toggle. When enabled:

1. The system still discovers the catalog.
2. The system still receives a real 402.
3. The system creates a random wallet from random bytes.
4. The random wallet has no funded USDC balance.
5. x402 attempts the same paid retry.
6. The facilitator/payment route rejects it, commonly with HTTP 402.
7. The backend emits a type=error event.
8. The browser marks the run failed and stores a failed ledger entry.

This is a real failure path using the same payment flow, not a simulated red message.

Browser ledger behavior

The incident ledger is browser-local. Storage key:

```
threshold.incident.ledger.v1
```

Each record contains:

- incident label
- outcome: resolved or failed
- reason
- cost
- cost label
- duration from first to last event
- display timestamp

Successful runs count toward “Incidents resolved without human input.” The average cost metric uses successful runs only. Failed runs have $0.000 USDC when payment does not settle. The duration label is intentionally “Time from failure to patch,” because the current flow receives a patch but does not apply and execute it in a sandbox.

HTTP/API reference

GET /api/catalog

Returns:

```
{ "apis": [ApiCatalogEntry, "..."] }
```

Used by the incident flow and catalog UI.

POST /api/agent/run-payment-flow

Returns SSE. This is the primary browser demo endpoint.

POST /api/resolve-incident

Protected by x402. Unpaid request should return 402. Paid request returns structured incident resolution.

POST /api/agent/run

Older/general agent route for summarize, code-review, and composed review-plus-summary flows. It is separate from the primary incident SSE demo.

GET /api/test

Small x402 gate used by pnpm test:pay to verify the base payment path.

POST /api/code-review and POST /api/summarize

Additional protected provider routes. They support the catalog and older/general agent workflows.

Environment contract

Required server startup variables:

- AVM_ADDRESS: valid Algorand receiver address for the test payment gate.
- PROVIDER_CODE_REVIEW_ADDRESS: valid provider receiver used by code review and incident resolution in the current prototype.
- PROVIDER_SUMMARIZE_ADDRESS: valid summarizer receiver.
- FACILITATOR_URL: GoPlausible facilitator URL.
- PORT: optional; defaults to 4021.
- X402_NETWORK: testnet for judging, or mainnet for explicit MainNet mode.
- X402_PRICE: optional; defaults to $0.001.
- AVM_MNEMONIC: server-only payer mnemonic.
- AVM_PAYER_ADDRESS: expected address derived from the mnemonic.
- GEMINI_API_KEY: server-only Gemini key.
- GEMINI_MODEL: optional Gemini model; defaults in code to gemini-3.5-flash.

Never expose or paste the values of mnemonic, API key, or private key in browser code, reports, issue trackers, or chat.

Commands and expected results

From repository root:

```
pnpm install
Set-Location .\server
pnpm typecheck
pnpm test
pnpm start
```

From the server directory:

```
pnpm test:pay
pnpm test:pay:resolve-incident
pnpm test:pay:code-review
pnpm test:pay:summarize
```

pnpm typecheck should exit successfully.

pnpm test runs the focused Node test file and verifies Lora URL generation, fixture consistency, sandbox patch application, and dependency-feed detection.

pnpm test:pay should show:

```
HTTP 402 Payment Required
Final HTTP 200 OK
Settlement success: true
Transaction ID: <fresh transaction>
```

A transaction ID must not be treated as valid solely because it looks like an Algorand ID. It should be queried or opened on Lora.

Verification facts and known limitations

Verified behavior:

- TypeScript typecheck passes.
- Focused automated tests pass.
- Direct TestNet x402 smoke test has produced HTTP 402 followed by HTTP 200 and Settlement success: true.
- Incident flow has produced a real GoPlausible settlement and structured results for its selectable scenarios.
- Transaction URLs use the required Lora TestNet format.
- Browser flow uses SSE and renders backend events.
- Underfunded wallet emits an error event.

Known limitations:

- Patches are verified in a temporary workspace, not applied to the user's real repository.
- AI response correctness is validated structurally, not formally proven.
- The payer is server-controlled for the prototype.
- Incident ledger is localStorage, not durable server storage.
- The incident fixture is controlled; the endpoint accepts general incident input but the main browser button sends the fixture.
- MainNet support exists in configuration but must not be claimed as the judging network when .env uses TestNet.
- The server must remain running for the browser. A terminal wrapper that kills long-running processes can produce a misleading process exit even when startup succeeded.

Agent operating instructions

When another agent uses this report, follow these rules:

1. Treat the source code as authoritative if this report is stale.
2. Do not claim a payment succeeded without a fresh settlement response or an explorer-verifiable transaction ID.

3. Do not call a UI label proof of backend behavior; inspect the event payload or server code.
4. Do not replace the real payment path with delays, scripted events, mock settlement objects, or fake transaction IDs.
5. Keep TestNet as the judging configuration unless the user explicitly changes the requirement.
6. Never print, expose, or commit .env secrets.
7. Preserve the single runIncidentPaymentFlow implementation for terminal and browser paths.
8. If changing the incident fixture, update the fixture consistency test and visible UI copy together.
9. If changing network or asset logic, run pnpm typecheck, pnpm test, and a real payment smoke test.
10. If changing SSE or browser event handling, test both funded and underfunded runs.
11. Distinguish “patch returned” from “patch applied and verified.” Bug patches are applied and tested in a temporary verification workspace; the user's real repository is not modified.
12. Before making claims, state which command, response, or explorer page provides evidence.

Judge-facing explanation

Threshold is an agent capability marketplace. A build agent that reaches a capability boundary queries the catalog, selects incident resolution or dependency vulnerability intelligence, receives a real HTTP 402 price requirement, signs a small USDC payment with the server-side Algorand payer, sends it through the GoPlausible facilitator, retries with payment proof, and receives a structured patch or security finding. The browser shows the real SSE event stream and settlement transaction. Bug patches are applied and verified in a temporary workspace; security scans use a clearly labeled OSV-compatible mock feed.

Final summary

The project proves this loop:

```
blocked agent
  -> catalog discovery
  -> capability selection
  -> HTTP 402
  -> Algorand USDC payment
  -> GoPlausible verification and settlement
  -> paid retry
  -> structured debugging patch
  -> live browser evidence
```

It is a credible prototype of machine-to-machine capability purchasing. Its central production gaps are wallet authorization, durable records, sandboxed patch application, and stronger verification of AI-generated changes.